Atributos de calidad en RECAUDO-T ampliado

Parte 2: Práctica – Resultados

Variante V5 – Ola de fraudes (Seguridad prioritaria, Disponibilidad secundaria)

Integrantes: Derick Rios, Santiago Siachoque, David Sanchez

1. Alcance de la práctica

La práctica implementa los escenarios seleccionados en la Parte 1 para la variante V5. Se implementaron tres tácticas, una por escenario y cada una de un atributo distinto, y se midió cada una contra una línea base sin la táctica.

Escenario	Atributo	Táctica implementada	Archivo principal
S1	Seguridad (prioritario)	Detectar ataques (uso imposible por velocidad) + revocar acceso (lista negra)	`recaudo/antifraude.py`
D2	Disponibilidad (secundario)	Reintentar (cola local persistente, espera exponencial, idempotencia)	`recaudo/validador.py`
M1	Modificabilidad	Diferir el enlace (tarifas en configuración, recarga en caliente)	`recaudo/tarifas_config.py`

1.1 Ajuste respecto a la Parte 1

En la Parte 1 se había elegido el escenario P1 con la táctica **Introducir concurrencia**. La regla 2 de esta práctica no acepta las tácticas ya implementadas en los talleres (Ping/Echo, Redundancia activa, Introducir concurrencia y Caché), así que P1 se reemplazó por **M1**, que en el árbol tiene prioridad (M, B) y una táctica permitida (diferir el enlace). El compromiso elegido en la Parte 1, **Seguridad vs. Desempeño**, se mantiene: se mide cuánta latencia agrega la detección de clones de S1. Se sigue cumpliendo que los tres escenarios son de atributos distintos y que uno (S1) es del atributo prioritario.

1.2 Sistema construido

Todo el sistema son procesos de software que se comunican por HTTP en la misma máquina. Lo real es el intercambio entre procesos y las mediciones; los usuarios, el atacante y la carga son scripts. Se usó solo la librería estándar de Python (`http.server`, `http.client`, `sqlite3`, `threading`): ninguna táctica viene de una librería que la traiga lista.

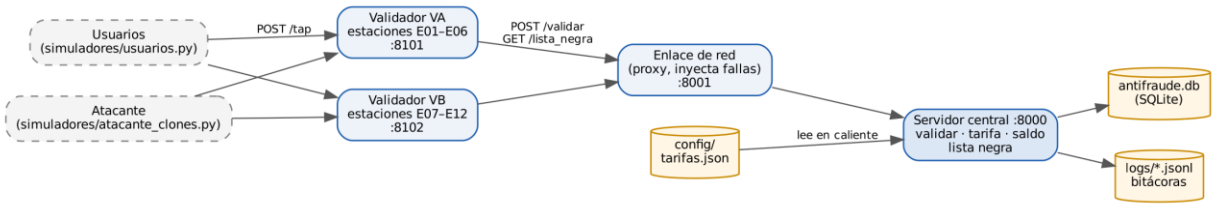


Figura 1. Procesos del sistema y cómo se comunican

Proceso	Puerto	Tipo	Responsabilidad
Servidor central (`recaudo.central`)	8000	Real	Valida pasajes, calcula la tarifa, descuenta saldo, motor antifraude y lista negra
Enlace (`recaudo.enlace`)	8001	Real	Proxy de red entre validadores y central; permite inyectar la caída del enlace
Validadores VA y VB (`recaudo.validador`)	8101, 8102	Real	Reciben los toques de tarjeta; cola local, reintento y lista negra local
Usuarios (`simuladores/usuarios.py`)	–	Simulado	Viajes legítimos con tiempos de desplazamiento realistas (20 km/h)

Proceso	Puerto	Tipo	Responsabilidad
Atacante (`simuladores/atacante_clones.py`)	–	Simulado	Usa tarjetas clonadas en estaciones y horas al azar
Generador de carga (`simuladores/carga.py`)	–	Simulado	Envía solicitudes a tasa fija y registra cada una en CSV

Ejecución con un solo comando: `python run.py` levanta el sistema; `python run.py experimentos` corre todos los experimentos y genera la carpeta `evidencias/`; `python run.py pruebas` corre las pruebas de regresión. Todos los números de este documento salen de `evidencias/resumen.json`.

2. Documentación de las tácticas

2.1 Táctica S1: Detectar ataques + revocar acceso (Seguridad)

Escenario

Campo	Contenido
Identificador	S1
Atributo	Seguridad
Fuente del estímulo	Delincuente con una copia clonada de una tarjeta legítima
Estímulo	La misma tarjeta se usa en dos validadores separados por una distancia imposible de recorrer en el tiempo transcurrido
Entorno	Operación normal, hora pico
Artefacto	Servicio de validación y motor antifraude
Respuesta	El sistema detecta el uso anómalo, rechaza la validación, pone la tarjeta en la lista negra, la distribuye a todos los validadores y avisa al titular
Medida de la respuesta	Detección en menos de 2 s; tarjeta bloqueada en el 100 % de los validadores en menos de 60 s; al menos 95 % de los clones detectados en una prueba con 1000 usos simulados
Prioridad	(A, A)

Cómo funciona

Una tarjeta clonada tiene el mismo número que la original, así que en el momento de leerla no se puede distinguir. Lo que la delata es el patrón de uso: el titular y el clon aparecen en estaciones distintas en momentos cercanos. Cada vez que llega una validación, el central busca el último uso de esa tarjeta, calcula la distancia entre las dos estaciones y la velocidad que habría necesitado el pasajero. Si supera 80 km/h (los buses no alcanzan esa velocidad promedio en la ciudad), la tarjeta se revoca: entra a la lista negra, la validación se rechaza y queda un registro en la bitácora. Los validadores descargan la lista negra cada 2 s, así que rechazan la tarjeta en el propio torniquete, incluso si después pierden la red.

Componentes

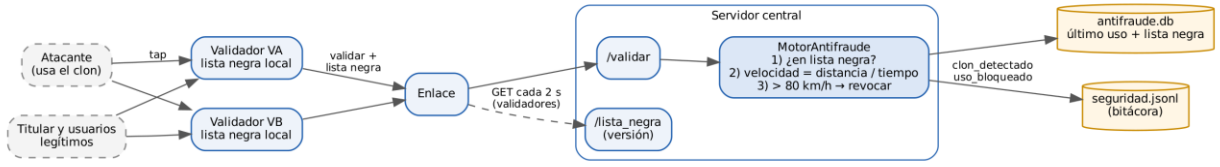


Figura 2. Procesos que participan en la táctica S1

Requisitos implementados

- Motor antifraude (`MotorAntifraude.evaluar`) que calcula distancia, tiempo entre usos y velocidad con las coordenadas de `config/estaciones.json`.
- Revocación: lista negra con número de versión; el central rechaza con HTTP 403 cualquier uso posterior de la tarjeta.
- Distribución: cada validador consulta `GET /lista_negra` cada 2 s y rechaza localmente las tarjetas revocadas.

- Persistencia: el último uso y la lista negra se guardan en SQLite (`antifraude.db`), así una caída o reinicio del central no borra las tarjetas revocadas (prueba `test_lista_negra_sobrevive_reinicio`).
- Bitácora `seguridad.jsonl` con un registro por detección (tarjeta, estaciones, distancia, segundos entre usos, velocidad, validador, acción y aviso al titular) y por cada uso bloqueado.
- Línea base: el mismo sistema con `ANTIFRAUDE=0`, que no evalúa nada.

Cómo medir

Métrica	De dónde sale el dato	Cómo se calcula
% de tarjetas clonadas detectadas	`seguridad.jsonl` (eventos `clon_detectado`)	Tarjetas clonadas que aparecen en la bitácora ÷ tarjetas clonadas del ataque
% de usos del clon bloqueados	`solicitudes.csv` (columna `rol` = clon)	Usos del clon con HTTP 403 ÷ usos del clon
Falsos positivos	`seguridad.jsonl`	Tarjetas legítimas que aparecen como detectadas
Tiempo de detección	`solicitudes.csv` (`ts_envio`) + bitácora (`ts`), unidas por `tx_id`	Hora del registro de la detección − hora en que el atacante envió el toque
Tiempo de propagación	Bitácora del central + `validador_VA.jsonl` y `validador_VB.jsonl`	Primera sincronización de cada validador con versión ≥ la de la revocación − hora de la detección; se toma el máximo de los dos
Reconstrucción	`seguridad.jsonl`	Detecciones con todos los campos (quién, dónde, cuándo, por qué, qué se hizo)

Experimento

Ataque de **200 tarjetas clonadas**: el titular hace 2 viajes reales y el clon hace 3 usos en estaciones y horas al azar dentro de la misma hora, en total **1000 usos de tarjetas atacadas** (como pide la ficha). Se mezclan con 500 usuarios legítimos (1.000 viajes) para medir falsos positivos. Todo se envía a los validadores a 100 toques/s. Se corrió igual sin la táctica y con ella.

Métrica	Sin la táctica	Con la táctica
Tarjetas clonadas detectadas	0 de 200 (0,0 %)	178 de 200 (89,0 %)
Usos del clon bloqueados	0 de 600 (0,0 %)	309 de 600 (51,5 %)
Usos del clon aceptados después de la detección	No aplica (no hay detección)	0
Falsos positivos (tarjetas legítimas bloqueadas)	0	0 de 500
Tiempo de detección (p50 / p95 / máx.)	No detecta	2,71 / 4,36 / 35,01 ms
Propagación a todos los validadores (p50 / máx.)	No aplica	0,99 / 2,00 s
Detecciones reconstruibles desde la bitácora	0 (no hay bitácora de seguridad)	178 de 178 (209 registros)

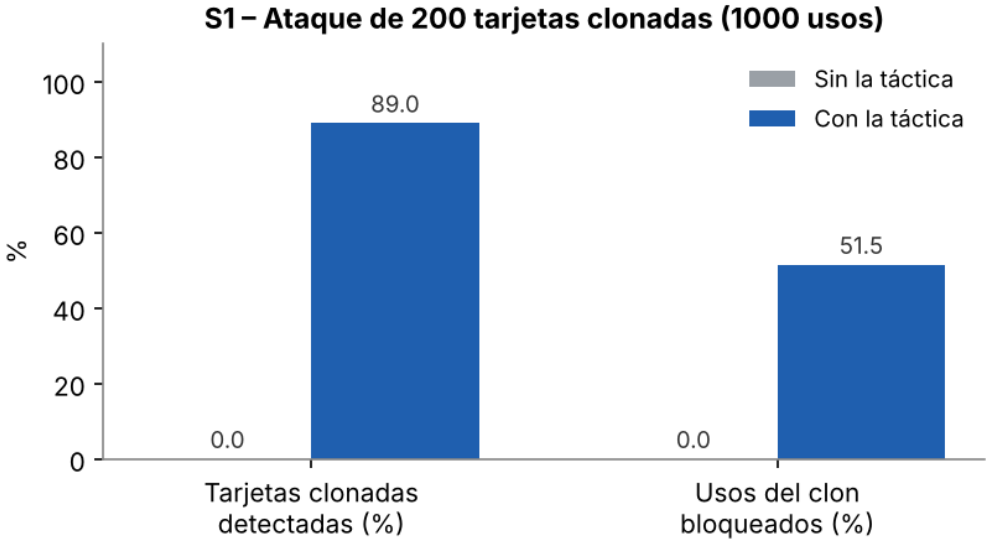


Figura 3. S1 – resultados del ataque sin la táctica y con ella

Sin la táctica, el sistema acepta todos los usos del clon y no deja rastro de que hubo un ataque. Con la táctica se detectó el 89,0 % de las tarjetas clonadas en unos 3 ms, la lista negra llegó a los dos validadores en máximo 2,0 s y ninguna tarjeta legítima fue bloqueada. Una vez revocada la tarjeta, el clon no pudo volver a usarla. También se bloquearon 264 usos del titular posteriores a la revocación: es el comportamiento esperado, porque la tarjeta comprometida se cancela y al titular se le avisa para reponerla.

La detección no llegó al 95 % porque algunos clones se usan pocas veces y en lugares y horas que sí serían posibles para una persona (por ejemplo, dos estaciones cercanas con 20 minutos de diferencia). Para entender ese límite se repitió el ataque variando cuántas veces usa el atacante la tarjeta clonada en una hora:

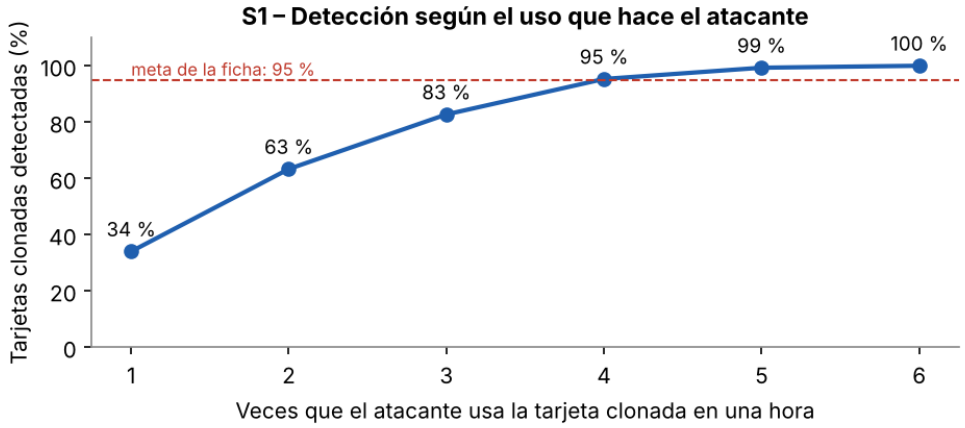


Figura 4. S1 – detección según cuántas veces usa el atacante el clon (150 tarjetas por punto)

Usos del clon por hora	Tarjetas detectadas	Usos del clon aceptados	Falsos positivos
1	34,0 %	128	0
2	63,3 %	215	0

Usos del clon por hora	Tarjetas detectadas	Usos del clon aceptados	Falsos positivos
3	82,7 %	242	0
4	95,3 %	251	0
5	99,3 %	235	0
6	100,0 %	215	0

Un clon que se usa una o dos veces pasa desapercibido con frecuencia, pero a partir de 4 usos por hora la detección supera el 95 %. Como el negocio del clonador es usar mucho la tarjeta antes de que la cancelen, el daño que alcanza a hacer queda acotado a muy pocos viajes. Evidencias: `evidencias/s1/` (CSV de solicitudes, bitácoras, `muestra_bitacora.jsonl` y `resultado.json`).

2.2 Táctica D2: Reintentar con cola local (Disponibilidad)

Escenario

Campo	Contenido
Identificador	D2
Atributo	Disponibilidad
Fuente del estímulo	Falla del enlace de comunicaciones
Estímulo	Un validador de bus pierde la conexión con el servidor central
Entorno	Modo degradado
Artefacto	Validador embarcado y servicio de sincronización
Respuesta	El validador sigue aceptando pasajes con las reglas locales, guarda las transacciones en una cola y las reenvía al recuperar la conexión
Medida de la respuesta	Operación local hasta 2 h; 100 % de transacciones sincronizadas en menos de 5 min tras la reconexión; 0 descuentos duplicados
Prioridad	(A, M)

Cómo funciona

Cuando el validador intenta enviar una validación y el enlace no responde, en vez de rechazar al pasajero acepta el pasaje en modo local (revisando primero la lista negra que tiene guardada), escribe la transacción en una cola en disco y sigue atendiendo. Mientras haya transacciones pendientes, las nuevas también van a la cola para no alterar el orden. Un hilo aparte intenta reenviar la primera transacción de la cola; si falla, espera cada vez más (0,25 s, 0,5 s, 1 s y como máximo 2 s) para no saturar la red. Cuando el enlace vuelve, vacía la cola en orden. Cada transacción lleva un identificador único (`tx_id`) y el central descarta los reenvíos repetidos, así reintentar nunca cobra dos veces.

Componentes

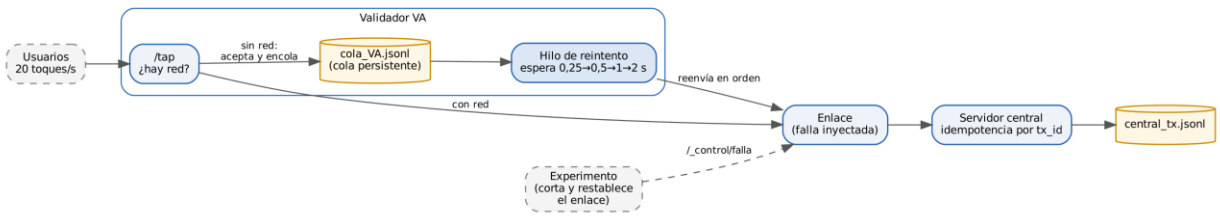


Figura 5. Procesos que participan en la táctica D2

Requisitos implementados

- Detección de la falla en el envío (conexión cortada, tiempo agotado o HTTP 502/503/504) y registro del evento `falla_detectada`.
- Cola persistente `cola_VA.jsonl`: si el validador se reinicia, recupera las transacciones pendientes.
- Hilo de reintento con espera exponencial y tope de 2 s; eventos `reconexion` y `cola_vacia` en la bitácora del validador.
- Idempotencia en el central: un diccionario de `tx_id` procesados devuelve la misma respuesta sin volver a cobrar.
- Proceso de enlace que corta las conexiones mientras la falla está activa (`POST /_control/falla`).
- Línea base: el mismo validador con `REINTENTO=0`, que rechaza el pasaje si no hay red.

Cómo medir

Métrica	De dónde sale el dato	Cómo se calcula
% de solicitudes exitosas	`toques.csv`	Toques con HTTP 200 ÷ toques enviados (total y durante la caída)
Tiempo de detección	`enlace.jsonl` (`falla_on`) y `validador_VA.jsonl` (`falla_detectada`)	Hora de detección − hora en que se cortó el enlace
Tiempo de recuperación	`enlace.jsonl` (`falla_off`) y `validador_VA.jsonl` (`reconexion`, `cola_vacia`)	Hora en que la cola quedó vacía − hora en que volvió el enlace
Transacciones perdidas	`toques.csv` y `central_tx.jsonl`	Toques aceptados cuyo `tx_id` no aparece cobrado en el central
Cobros duplicados	`central_tx.jsonl`	`tx_id` con más de un cobro
Carga adicional	`muestras_cola.csv` y estadísticas del enlace	Intentos cortados durante la caída y ritmo de reenvío al reconectar

Experimento

El validador VA recibe 20 toques/s durante 120 s (2.400 toques). En el segundo 30 se corta el enlace y en el segundo 90 se restablece: **60 s de caída con 1.200 toques**. Un bus real valida unas 10 personas por minuto, así que ese volumen equivale a unas **2 horas** de operación sin red de un bus, que es lo que exige la ficha.

Métrica	Sin la táctica	Con la táctica
Toques aceptados (total)	1.200 de 2.400 (50,0 %)	2.400 de 2.400 (100,0 %)
Toques aceptados durante la caída	0,1 %	100,0 %
Pasajeros rechazados por falta de red	1.200	0
Tiempo de detección de la falla	1 ms	1 ms
Reconexión tras volver el enlace	No aplica	1,86 s
Sincronización completa de la cola	No aplica	3,55 s (1.235 transacciones)
Transacciones perdidas	0	0
Cobros duplicados	0	0

Métrica	Sin la táctica	Con la táctica
Reenvíos repetidos descartados (prueba de idempotencia)	No aplica	100 de 100
Latencia del toque durante la caída (p95)	Rechazado	2,68 ms

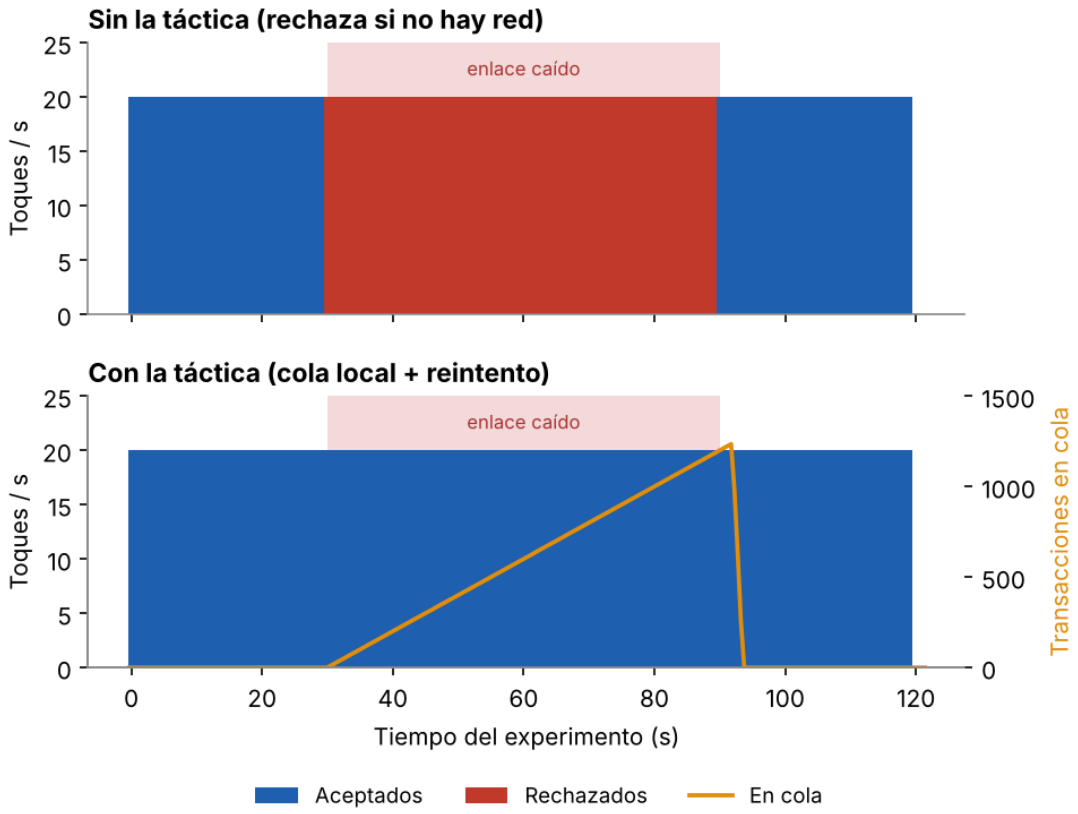


Figura 6. D2 – toques aceptados y rechazados por segundo; la franja roja es la caída del enlace

Sin la táctica, durante toda la caída el validador rechaza a los pasajeros: se pierde la mitad de las validaciones del experimento. Con la táctica el pasajero no nota la falla (el toque se responde en 2,7 ms en el p95), la cola llega a 1.235 transacciones y se vacía 3,5 s después de que vuelve el enlace, sin perder ni duplicar ningún cobro. Para probar la idempotencia se reenviaron a propósito 100 transacciones ya cobradas (como si el validador hubiera perdido la confirmación) y el central las descartó todas.

Costo de reintentar: durante la caída el validador hizo 33 reintentos, gracias a la espera exponencial (sin ella, con un reintento cada 50 ms, serían unos 1200). Al reconectar, el central recibe de golpe toda la cola: el ritmo de llegada subió hasta unas 782 validaciones/s, frente a las 20/s normales. Con muchos buses reconectando a la vez, esa ráfaga es un riesgo para el desempeño del central.

2.3 Táctica M1: Diferir el enlace (Modificabilidad)

Escenario

Campo	Contenido
Identificador	M1
Atributo	Modificabilidad

Campo	Contenido
Fuente del estímulo	Alcaldía
Estímulo	Cambia el valor del pasaje para estudiantes en hora valle (en la prueba, además, crea un tipo de usuario nuevo)
Entorno	Operación normal
Artefacto	Módulo de cálculo de tarifas
Respuesta	El cambio se configura y entra en vigor sin modificar código ni reiniciar el servicio
Medida de la respuesta	Máximo 1 día-persona de esfuerzo; 0 redespliegues; 0 regresiones en las pruebas de tarifas existentes
Prioridad	(M, B)

Cómo funciona

Las tarifas no están escritas en el código: están en una tabla dentro de `config/tarifas.json` (tipo de usuario, franja y transbordos). El servidor central lee esa tabla en tiempo de ejecución. Cada 0,5 s, como máximo, revisa si el archivo cambió; si cambió, lo valida (que estén todos los campos y que los valores sean enteros no negativos) y lo carga sin detenerse. Si el archivo nuevo tiene un error, conserva la versión anterior y lo deja anotado en la bitácora, así un error de digitación no deja a la ciudad sin cobro.

Componentes

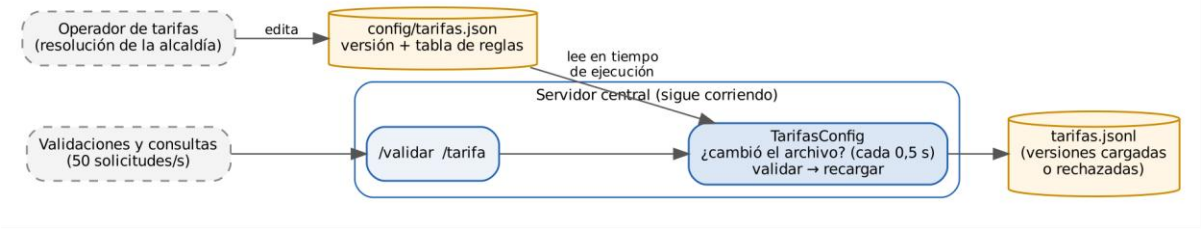


Figura 7. Procesos que participan en la táctica M1

Requisitos implementados

- Motor `TarifasConfig` que lee la tabla de tarifas en tiempo de ejecución y la recarga cuando cambia la fecha de modificación del archivo.
- Validación de la configuración antes de aplicarla; si es inválida se conserva la anterior (prueba `test_configuracion_invalida_se_rechaza_y_se_conserva_la_anterior`).
- Bitácora `tarifas.jsonl` con cada versión cargada o rechazada.
- Pruebas de regresión: la configuración da exactamente las mismas tarifas que el código original en las 24 combinaciones de tipo, franja y transbordos.
- Línea base: `TarifasCodigo`, con las tarifas en una cadena de `if/elif`, activable con `TARIFAS_MODO=codigo`.

Cómo medir

Métrica	De dónde sale el dato	Cómo se calcula
Archivos y líneas modificadas	`cambio.diff` de cada variante	Conteo de archivos y de líneas agregadas y borradas en el diff
¿Hubo que reiniciar?	Script del experimento y `central.out`	Sí si el cambio solo entra en vigor reiniciando el proceso

Métrica	De dónde sale el dato	Cómo se calcula
Tiempo hasta que la tarifa está vigente	`consultas.csv`	Primera consulta que devuelve el valor nuevo − momento del cambio
Solicitudes fallidas durante el cambio	`consultas.csv`	Consultas sin respuesta (conexión rechazada) mientras se aplica el cambio
Regresiones	`consultas.csv` y pruebas unitarias	Tarifas que no debían cambiar y cambiaron

Experimento

Cambio controlado (resolución ficticia de la alcaldía): el pasaje de estudiante en hora valle baja de 2100 a 1950 y se crea el tipo de usuario "discapacidad" (1600 en pico, 1500 en valle). Se hizo el mismo cambio en las dos versiones mientras el central atendía 50 consultas de tarifa por segundo; el cambio se aplicó en el segundo 6 de 20.

Métrica	Sin la táctica (código)	Con la táctica (configuración)
Archivos de código modificados	1	0
Archivos de configuración modificados	0	1
Líneas agregadas / borradas	5 / 1	4 / 3
¿Hubo que reiniciar o redesplegar?	Sí	No
Tiempo con el servicio caído	79 ms	0 ms
Consultas fallidas durante el cambio	4	0
Tiempo hasta que la tarifa nueva responde	0,14 s	0,14 s
Regresiones en tarifas no modificadas	0	0

El tamaño del cambio es parecido en los dos casos (unas pocas líneas), pero su naturaleza es distinta. Sin la táctica se edita lógica de negocio en Python y hay que reiniciar el proceso: en el experimento eso dejó el servicio caído 79 ms y 4 consultas sin respuesta a solo 50 solicitudes/s. En producción, además, un cambio de código pasa por revisión, pruebas y despliegue en todas las instancias, y con cambios casi mensuales eso se repite todo el año. Con la táctica solo se edita una tabla de datos, el servicio nunca se detiene y la tarifa nueva rige en 0,14 s, sin consultas fallidas. Los diffs exactos de cada variante están en `evidencias/m1/*/cambio.diff`.

3. Medición de los atributos

Cada atributo se midió como indica la guía de la práctica. Todo número del documento tiene un archivo de evidencia en el repositorio.

Atributo	Cómo se midió	Evidencia
Seguridad (S1)	Ataque de 200 tarjetas clonadas (1000 usos) antes y después de la táctica: % de ataques bloqueados, tiempo de detección y de reacción, falsos positivos y si la bitácora permite reconstruir lo ocurrido	`evidencias/s1/linea_base/` y `con_tactica/`: `solicitudes.csv`, `seguridad.jsonl`, `validador_*.jsonl`, `resultado.json`; barrido en `evidencias/s1/barrido/`
Disponibilidad (D2)	Falla inyectada (corte del enlace por 60 s): tiempo de detección, tiempo de recuperación y % de solicitudes exitosas; además, pérdidas y duplicados	`evidencias/d2/*/`: `toques.csv`, `muestras_cola.csv`, `enlace.jsonl`, `validador_VA.jsonl`, `central_tx.jsonl`
Modificabilidad (M1)	Cambio controlado de tarifas sin y con la táctica: archivos y líneas modificadas, reinicio, tiempo del cambio, solicitudes fallidas y regresiones	`evidencias/m1/*/cambio.diff`, `consultas.csv`, `resultado.json`; `tests/test_tarifas.py`
Desempeño (compromiso)	Carga de 250 validaciones/s durante 30 s, 3 repeticiones por condición: latencia (p50, p95, p99), throughput, jitter y solicitudes no procesadas	`evidencias/compromiso/*_repN/latencias.csv`, `corridas.csv`, `resultado.json`

Definiciones usadas. Latencia: tiempo entre el envío de la solicitud y la respuesta completa, medido por el generador de carga. Percentiles con interpolación lineal (igual que PERCENTIL.INCLUIDO de Excel). Jitter: promedio de la diferencia absoluta entre latencias consecutivas. Throughput: solicitudes respondidas ÷ duración de la corrida. Las horas de la bitácora son hora real del sistema (epoch, segundos).

4. Experimento del compromiso: Seguridad vs. Desempeño

La detección de clones (S1) se ejecuta dentro de cada validación, en el camino crítico: antes de cobrar, el central tiene que leer el último uso de la tarjeta, calcular la velocidad y guardar el nuevo uso. La hipótesis de la Parte 1 era que esto mejora la seguridad y empeora la latencia de validación.

4.1 Diseño

- Misma carga en todas las condiciones: viajes legítimos enviados directo al central a **250 validaciones/s durante 30 s** (7500 por corrida).
- Tres condiciones: **sin detección** (línea base), **con detección** tal como quedó implementada (estado en SQLite para que sobreviva a reinicios) y, como referencia, **con detección solo en memoria**.
- 3 repeticiones por condición, en orden rotado, para que el ruido de la máquina afecte por igual a todas (en total 22 500 validaciones por condición).
- Además, un microbenchmark de la función `evaluar()` fuera de HTTP para aislar su costo.

4.2 Resultados

Métrica	Sin detección	Con detección (SQLite)	Con detección (memoria)
Latencia media	1,436 ms	1,517 ms	1,544 ms
Latencia p50	1,350 ms	1,409 ms	1,443 ms
Latencia p95	1,915 ms	2,166 ms	2,098 ms
Latencia p99	3,150 ms	5,597 ms	3,604 ms
p95 por repetición	1,92 / 1,82 / 1,99 ms	2,33 / 2,35 / 1,58 ms	2,11 / 2,16 / 2,02 ms
Jitter	0,252 ms	0,313 ms	0,284 ms
Throughput	247,2 val/s	247,2 val/s	247,2 val/s
Validaciones no procesadas	0	0	0
Costo de `evaluar()` (microbenchmark)	0	43,8 µs	2,0 µs

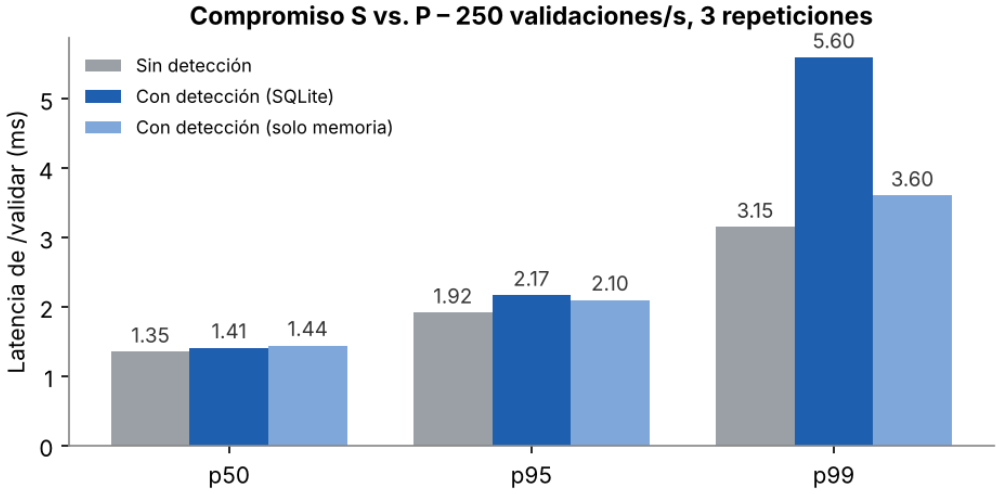


Figura 8. Latencia de validación en las tres condiciones

4.3 Análisis

El compromiso se confirma: con la detección activa, la latencia p95 de validación sube de 1,92 a 2,17 ms (**+0,25 ms, 13,1 %**), la media sube 0,08 ms, el p99 pasa de 3,15 a 5,60 ms y el jitter aumenta de 0,252 a 0,313 ms. En las repeticiones 1 y 2 la detección subió la media, el p95 y el p99. En la repetición 3 toda la corrida fue más rápida de lo normal (p50 de 1,0 ms frente a 1,3–1,6 ms en las otras ocho corridas), lo que apunta a ruido de la máquina; aun así su p99 fue el más alto de esa repetición. Por eso se reportan las tres repeticiones juntas y por separado. El throughput no cambia porque a esta carga el central no está saturado.

La comparación con la versión en memoria explica de dónde sale el costo: el algoritmo de detección cuesta solo 2,0 µs, prácticamente nada, mientras que guardar el estado en SQLite cuesta 43,8 µs por validación y se hace con un candado, así que en momentos de muchas solicitudes simultáneas unas esperan a otras (eso es lo que dispara el p99). Ese guardado es necesario por seguridad: sin él, un reinicio del central borraría la lista negra y los clones ya detectados volverían a funcionar.

4.4 ¿Es aceptable el costo para la variante V5?

Sí. En la variante V5 la seguridad es el atributo prioritario y el desempeño no es ni siquiera el secundario. El costo medido es de décimas de milisegundo, muy lejos de la meta de la Parte 1 para validación en hora pico (p95 menor a 300 ms): con la detección activa el p95 es de 2,17 ms. Como la sección que guarda el estado tarda unos 44 µs y se ejecuta de una en una, un nodo central alcanzaría hasta unas 22.900 detecciones por segundo antes de que ese paso sea el cuello de botella, holgado para las 1500 validaciones/s de hora pico. A cambio se bloquea el uso de las tarjetas clonadas en milisegundos. Si la flota crece mucho (variante V4), el punto a vigilar es justamente ese candado: se podría repartir el estado por número de tarjeta en varias bases.

5. Verificación de escenarios

Escenario	Medida esperada (de la ficha)	Medida obtenida	¿Se cumple?	Si no se cumple, ¿por qué?
S1	Detección en menos de 2 s	35,0 ms (máximo)	Sí	
S1	Tarjeta bloqueada en el 100 % de los validadores en menos de 60 s	178 de 178 detecciones (100 %) en máx. 2,0 s	Sí	
S1	Al menos 95 % de los clones detectados con 1000 usos simulados	89,0 % (0 falsos positivos)	No	Un clon usado pocas veces en lugares y horas plausibles no produce un viaje imposible. Con 4 o más usos por hora se supera el 95 % (barrido). Para cerrar la brecha se necesita complementar con autenticación criptográfica de la tarjeta o con reglas adicionales en el motor (escenario M2).
D2	Operación local hasta 2 h	60 s sin red con 1.200 toques (≈ 2 h de un bus), 100 % aceptados	Parcial	Se probó el volumen equivalente a 2 h, no la duración real. La cola en disco no tiene límite de tiempo, pero no se ejecutó una caída de 2 horas.
D2	100 % sincronizadas en menos de 5 min tras la reconexión	100 % en 3,5 s, 0 perdidas	Sí	
D2	0 descuentos duplicados	0 duplicados; 100 reenvíos repetidos descartados	Sí	
M1	Máximo 1 día-persona de esfuerzo	4 líneas agregadas y 3 borradas en 1 archivo de datos; vigente en 0,14 s	Sí	El esfuerzo humano no se cronometró; el cambio es editar 4 líneas de una tabla, lo que toma minutos.
M1	0 redespliegues	0 reinicios, 0 consultas fallidas	Sí	
M1	0 regresiones en las pruebas de tarifas existentes	0 regresiones; 7 de 7 pruebas unitarias OK	Sí	
S vs. P	p95 de validación menor a 300 ms con la detección activa (meta de P1)	2,17 ms a 250 val/s	Sí	Se probó a 250 val/s por los límites de la máquina; no se probaron 1500 val/s.

Resumen. De las 10 medidas, 8 se cumplen, una se cumple parcialmente (la duración de 2 h de D2, que se verificó por volumen) y una no se cumple (el 95 % de detección de S1 con pocos usos del clon). Este último resultado no se ocultó porque muestra el límite real de detectar clones solo por velocidad, y da la razón para la siguiente mejora de seguridad.

6. Entregables

6.1 Repositorio

Carpeta o archivo	Contenido
`run.py`	Comando único: `python run.py` (sistema), `python run.py experimentos`, `python run.py pruebas`
`recaudo/`	Central, validador, enlace y las tres tácticas (`antifraude.py`, `validador.py`, `tarifas_config.py`); línea base en `tarifas_codigo.py`
`simuladores/`	Usuarios, atacante y generador de carga
`experimentos/`	Un script por experimento (`exp_s1.py`, `exp_d2.py`, `exp_m1.py`, `exp_compromiso.py`) y `graficas.py`
`config/`	`estaciones.json` y `tarifas.json`
`tests/`	Pruebas de regresión de tarifas y antifraude
`docs/diagramas/`	Diagramas de componentes (fuente `.dot` y `.png`)
`docker-compose.yml`	Alternativa para levantar el sistema con contenedores

6.2 Evidencias

Archivo	Qué demuestra
`evidencias/resumen.json`	Todos los números de este documento
`evidencias/s1/*/solicitudes.csv`	Cada uso de tarjeta del ataque, con su resultado y latencia
`evidencias/s1/con_tactica/seguridad.jsonl`	Bitácora de seguridad: cada clon detectado y cada uso bloqueado
`evidencias/s1/barrido/barrido_usos_clon.csv`	Detección según el número de usos del clon
`evidencias/d2/*/toques.csv`, `muestras_cola.csv`	Cada toque y el tamaño de la cola cada 0,5 s
`evidencias/d2/*/enlace.jsonl`, `validador_VA.jsonl`, `central_tx.jsonl`	Momento de la falla, detección, reconexión, vaciado de la cola y cobros
`evidencias/m1/*/cambio.diff`	El cambio exacto hecho en el código (línea base) y en la configuración (táctica)
`evidencias/m1/*/consultas.csv`	Consultas de tarifa durante el cambio
`evidencias/compromiso/*_repN/latencias.csv`	Latencia de cada validación en las 9 corridas
`evidencias/graficas/`	Las gráficas de este documento

Para repetir todas las mediciones: `python run.py experimentos` (unos 15 minutos). Los valores pueden variar un poco según la máquina, pero las conclusiones se mantienen.